

Capability-Conditioned Scaling-Down Laws for LLM Compression

Anonymous authors

Paper under double-blind review

Abstract

We systematically investigate whether information observable before compression can predict capability-conditioned loss after pruning, quantization, and distillation. The endpoint is a per-capability conditional loss (math, code, multi-hop QA) measured relative to the dense source or, for distillation, the initial student; the inputs are the source size, its pretraining tokens, its dense capability losses, and the intervention setting. We separate transfer across source-model states from prediction at unseen intervention settings, evaluate each candidate against baselines with the same information budget, and freeze predictions before new sources are measured. On a heterogeneous panel of 12 public models from five series, responses are strongly capability- and series-specific: QA is the least-damaged capability in Qwen3 and OLMo-3-32B and the most-damaged one in Gemma at mild pruning, and 3-bit quantization collapses 12 of 14 models but not OLMo-3-32B. On a controlled Pythia panel, the dense capability vector carries the prediction; source-conditioned pruning predictors beat source-free curves at seen densities and at interpolated densities on the development sources, match them at a new in-range size, and fail at density 0.55 and at a source five times larger; a shared power form adds nothing over a per-density regression. Quantization gains appear only in the collapse regime and depend on the fitting protocol; for distillation a reuse-count coordinate helps QA on an unseen pool with a budget-dependent bias, and a constant is best for math and code. The results identify conditional predictive relationships and their tested limits, providing an empirical basis for developing capability-conditioned scaling-down laws.

1 Introduction

Deployed language-model agents are specialists. A coding agent depends on code synthesis and tool use, a retrieval assistant on multi-hop question answering, and neither needs frontier-scale generality while both pay for every token of inference. The practical question is which model covers a workload’s capability profile at the lowest cost, and the usual answer starts from a strong dense source and scales it down by pruning, quantization, or distillation, often with recovery training. Compression does not shrink capabilities uniformly. At the same size or bit-width, different methods and different source families preserve different capabilities, improve some while collapsing others, and hit cliffs at different strengths (§5.1). Aggregate perplexity and average benchmark scores hide this, so choosing a route today means running each candidate configuration through capability benchmarks.

This paper asks whether that search can be predicted from what is observable before compression. We fix a per-capability conditional loss as the endpoint and write every prediction relative to a reference measured beforehand: the dense source for pruning and quantization, the initial student for distillation (§3). The admitted inputs are the source size, its pretraining token count where disclosed, its dense capability losses, and the intervention setting. The question is then concrete. Can pre-compression model-state information and intervention settings predict capability-conditioned loss, and over which source and intervention ranges do those predictions transfer?

Answering it requires separating three things that a single accuracy number would blur. A predictor can fail because the source information is useless, because the functional form is wrong, or because the test lies outside the range in which anything was fit. We therefore test two axes separately, transfer across source-model states at a fixed intervention and prediction at unseen intervention settings, and we compare every candidate against baselines that use the same information: a source-free strength curve or a per-configuration constant asks whether the source state helps at all, a no- D_0 variant asks what training tokens add, and same-input alternatives ask whether a particular form adds anything (§4). New-source tests are frozen prospectives: forms, coefficients, and predictions are committed before the compressed model is measured, and a failed prospective keeps its label.

The evidence has three layers (§5). On a heterogeneous panel of 12 public models from five series, the capability response to compression is strongly capability- and series-specific: QA is the least-damaged capability in Qwen3 and OLMo-3-32B, where it improves under moderate pruning, and the most-damaged one in the Gemma series at mild densities; 3-bit quantization collapses 12 of 14 models but not OLMo-3-32B. On a controlled Pythia panel, the dense capability vector is the input that carries prediction, training tokens add little beyond it, and source-conditioned pruning predictors beat source-free curves at seen densities and at interpolated densities on the development sources, mostly for math and code, and match them at a new in-range size; a shared power form adds nothing over a per-density regression with a fixed interpolation rule. The same predictors fail at density 0.55 and at a source five times larger than the development range, in both arms. Quantization gains appear only in the collapse regime and depend on the fitting protocol. For distillation, a reuse-count coordinate helps QA on an unseen data pool with a budget-dependent bias, and a constant is best for math and code. Method-agnostic laws in storage or active-parameter ratios fail a matched-storage test.

Contributions. (1) A capability-conditioned prediction problem with an explicit admitted-information budget, a common loss interface across three compression methods, and a two-axis validation design that separates source transfer from configuration transfer (§3, §4). (2) Candidate relations for each arm, compared on the same inputs against same-budget baselines, with frozen prospective tests on new sources and their measured range of validity (§5.2–5.4). (3) An empirical account of the conditions under which transferable prediction is possible and of its failure boundaries, with reproducible measurements, registers, and evaluation code (§5.5). Dense-reference scaling laws are prior work that we use, the Fisher implementation is infrastructure, and method selection is discussed but not validated.

2 Related Work

Compression scaling laws with a loss endpoint. Most compression laws predict an aggregate loss: sparsity as a multiplicative factor on the size term (Frantar et al., 2024), precision through an effective-parameter count with a degradation term that grows with pretraining tokens (Kumar et al., 2025), formats used during training through one capacity scalar (Panferov et al., 2025), and a distilled student through a broken power law in teacher loss (Busbridge et al., 2025). Closest to our pruning arm, the P^2 law (Chen et al., 2025) predicts the post-training loss of a pruned model from its size and loss before pruning, the pruning rate, and the post-training tokens, validated on held-out tokens, sizes, and rates; source-state conditioning is thus not new in itself. None of these fit or validate a form on per-capability measurements (Busbridge et al. (2025) report downstream accuracy only as a proxy check).

Laws with downstream endpoints. Sengupta et al. (2025) fit pruning laws on test loss and averaged zero-shot accuracy against the compression ratio and recovery tokens; the revised version of the same work adds rolling hold-out over pruning ratios, zero- and one-shot transfer to unseen models, and per-task exponents. Its inputs are the unpruned score and the pruning ratio, with no source-state or training-history term, and its endpoint is an accuracy or an inverse log-perplexity. Bit-width laws for zero-shot accuracy exist at large scale (Dettmers & Zettlemoyer, 2023), and Zhou et al. (2026) stratify post-training

quantization laws into memorization, application, and reasoning strata for one quantizer, validated on a held-out larger model and a second family; those laws carry no source-state input and no recovery axis. Task-specific distillation has been studied as curves over dataset size, compression ratio, and supervision format without a fitted law (Ghita et al., 2026). Relative to this line, the present study keeps a per-capability conditional loss as the endpoint for three methods at once, admits the dense capability vector and the training-token count as inputs on a panel that separates size from training stage, and reports frozen tests against baselines with the same information, including the tests that fail.

Predictability of downstream capabilities. Downstream accuracy appears predictable mainly through continuous log-likelihood measurements: emergent jumps are largely metric artifacts (Schaeffer et al., 2023), accuracy-space prediction degrades through information-destroying transforms (Schaeffer et al., 2025), and successful pipelines fit loss-space laws with sigmoid or exponential links (Gadre et al., 2025; Bhagia et al., 2025; Owen, 2024; Ruan et al., 2024); even then, linear loss-to-downstream relations hold in a minority of settings (Lourie et al., 2025). The measurement layer of §3.3 follows these lessons.

Capability-selective effects of compression. Pruning disproportionately harms knowledge-intensive tasks (Jaiswal et al., 2024), low-bit quantization disproportionately harms reasoning (Liu et al., 2025), small-magnitude weights encode hard-task knowledge whose removal is not recovered by further training (Yin et al., 2024), and selective low-rank truncation can improve specific behaviors (Sharma et al., 2024). These observations motivate a capability-vector endpoint; the present paper adds per-model, per-configuration measurements across families and a prediction test of the resulting heterogeneity.

Gradients, curvature, and law forms. Second-order weight importance underlies classical and modern pruning (LeCun et al., 1989; Hassibi & Stork, 1992; Frantar & Alistarh, 2023; Sun et al., 2024), Fisher-aligned subspaces improve compression choices (Shihab et al., 2026), and gradient structure localizes skills (Gur-Ari et al., 2018; Zhao et al., 2024; Michaud et al., 2023; Ilharco et al., 2023); we use capability-conditional gradient and Fisher statistics only as mechanism evidence (Appendix B), never as inputs. Surveys document the fragility of purely fitted scaling-law forms (Li et al., 2025; Besiroglu et al., 2024); the forms tested here are low-dimensional candidates compared by held-out error and kept only where they predict.

3 Problem and Measurement

3.1 The prediction problem

We fix a capability set $\mathcal{C}$ (here mathematical reasoning, code generation, and multi-hop question answering) and study one quantity per capability, the conditional loss L_c of Eq. equation 2 below. For a compression method m applied to a model, the target of prediction is the post-compression capability vector, written relative to a reference that is measured before the intervention:

$$\hat{L}_{m,c} = L_{c,\text{ref}(m)}^{\text{obs}} + \hat{\Delta}_{m,c}(\mathbf{x}, \mathbf{z}, \boldsymbol{\theta}_m). \quad (1)$$

The reference differs by arm. For pruning and quantization it is the dense source model itself, so $\hat{\Delta}_{m,c}$ predicts the damage $\Delta L_c = L_c(\text{compressed}) - L_c(\text{dense})$. For distillation it is the initial student, so the target is the signed transfer response $\delta_c = L_c(S_{\text{KD}}) - L_c(S_0)$ of the student to the teacher traces; the gap between teacher and student size never enters this quantity. The three arms therefore share an endpoint and an evaluation protocol, and nothing else is assumed to be shared: whether one functional form covers several arms is a question the data have to answer (§5.5).

The inputs fall into three groups. The state vector $\mathbf{x}$ holds what is observable about the source (or the student) before the intervention: the parameter count N_0 , counted throughout as the number of entries of the transformer weight matrices (attention and MLP projections; embeddings, output head, biases, and norms excluded), the pretraining token count D_0

Budget	Information available	Cost that must be disclosed
K0, metadata only	N_0 , public D_0 , architecture, intervention configuration	availability of metadata; D_0 may be undisclosed
K0 + dense anchor	the above plus the dense capability losses $\mathbf{L}_0$ measured once before compression	probe count, one forward pass per probe, native loss units
K1	the above plus one pre-specified measurement of the target at one compression configuration	choice of the calibration configuration and its compression cost
Oracle diagnostic	post-hoc rescaling or recalibration on the target outcomes	none; used only to diagnose form misfit, never as a prediction

Table 1: Information budgets. Unknown D_0 stays unknown; any handling of missing inputs is fixed on the development panel and never inferred from the target response. The dense reference laws of prior work predict $\mathbf{L}_0$ from (N_0, D_0) ; here $\mathbf{L}_0$ is measured, and predicting it is a separate task that this paper does not attempt.

where it is disclosed, and the measured dense capability vector $\mathbf{L}_0 = (L_{0,\text{math}}, L_{0,\text{code}}, L_{0,\text{qa}})$. Family identity and per-model fitted amplitudes are not basic parameters; where a model uses them, we say so. The shared descriptors $\mathbf{z}$ are the quantities on which arms can be compared: retained density, bit-width, and the processed and unique training tokens of a distillation run. The method controls θ_m are the per-arm settings held fixed by protocol (pruning criterion, quantizer, student size, supervision recipe). We do not invent shared variables for an arm that has none.

3.2 Information budgets

Every predictor in this paper is labeled by the information it may use. Table 1 defines four budgets; all headline predictions use the second one, and the first is the reference for what training tokens add to metadata alone.

3.3 Operational loss definition

Each capability c has a probe set $\mathcal{P}_c$ of prompt and reference pairs drawn once, with a fixed seed, from held-out benchmark splits: MATH-500 for math, MBPP for code, and 2WikiMultiHopQA for QA as the primary benchmarks. The observable is the token-weighted conditional cross-entropy of the reference completion given the prompt,

$$L_c(w) = \frac{1}{T_c} \sum_{(x,y) \in \mathcal{P}_c} \sum_{t=1}^{|y|} -\log p_w(y_t \mid x, y_{<t}), \quad T_c = \sum_{(x,y) \in \mathcal{P}_c} |y|, \quad (2)$$

with prompt tokens masked, sequences truncated to 1024 tokens, and the model’s own tokenizer throughout. The same definition, the same 64 measurement probes per capability, and the same scoring code are used for every model state that is compared within a model. We report native nats per token. Within-model differences ΔL_c are computed on identical token sequences and are the only quantities that enter predictors; absolute levels across tokenizers are not placed on one numerical scale, and where families are compared the comparison is made within family or in per-byte units in the appendix.

Three boundaries of this observable are stated once here. First, it is the negative log-likelihood of a fixed reference, which tracks task success closely for code and math on these benchmarks but is not a task-success measure; a QA loss can fall while exact-match accuracy does not rise (Appendix A.1). Second, the reference distribution is the full solution; direct-answer and given-reasoning-answer variants were measured for a subset of models and differ from it (Appendix A.1), so the numbers here are specific to the full-solution reference. Third, capability specificity is supported by a prediction test, not assumed: within-capability responses on an independent secondary benchmark (SVAMP, HumanEval, TriviaQA) are predicted from the primary benchmark better than by cross-capability responses for math

and QA, whereas for code the within-capability gain was not detected at the available resolution (Appendix A.1). Gradient and Fisher statistics of the dense model appear only as mechanism evidence in the appendix; they are never prediction inputs.

4 Study Design and Predictive Models

4.1 Panels

Three kinds of panel serve three purposes (Appendix Table 4). The heterogeneous panel of finished public models tests how far a relation carries across families whose training history, data, and architecture all differ at once; it establishes the phenomena and the range of responses, and its family and data provenance are confounded by construction. The controlled Pythia panel varies the source state along known axes, size and pretraining stage, with architecture and data held fixed, and is the development set of every predictor. The distillation panels vary the student, the data pool, and the exposure with the teacher fixed.

4.2 Predictors that were actually tested

Every predictor below was fit on the development set only, with its standardization constants, coefficients, and keep rules committed to a register file before any test measurement (paths in Appendix A.4). Parameter counts are per capability.

Configuration-indicator source regression (source axis). At each intervention configuration q , a density or a bit-width, a separate linear model in the standardized inputs, $\widehat{\Delta L}_c(q) = \beta_{c,q} \cdot (1, \log \tilde{N}_0, \tilde{L}_{0,c}, \log \tilde{D}_0)$, four coefficients per configuration (16 for four densities). It predicts a new source state at a seen configuration and is not a law over the intervention axis. Its same-input comparator drops $\log \tilde{D}_0$ (three coefficients per configuration). This is the predictor frozen for the step-96k tests (register v38) and, per bit-width, the quantization predictor throughout.

Shared power form (pruning, configuration axis). $\widehat{\Delta L}_c(\mathbf{x}, d) = A_c(\mathbf{x}) ((1 - d)/0.3)^{\gamma_c}$ with $A_c(\mathbf{x}) = \beta_c \cdot (1, \log \tilde{N}_0, \tilde{L}_{0,c}, \log \tilde{D}_0)$, five parameters, fit on the nine development states at $d \in \{0.9, 0.8, 0.7, 0.6\}$ (108 rows; register v40, refrozen with the same form in v49). It is the headline candidate for unseen densities.

Same-input alternatives for pruning. Three forms use exactly the inputs of the shared power form and a different strength structure: A1, the power form with γ_c fixed to one (four parameters); A2, the configuration-indicator regression at the four development densities followed by a fixed rule, linear interpolation between adjacent predicted anchors and linear extension beyond the boundary pair; and a continuous two-term polynomial in $s = 1 - d$ with source-dependent coefficients. They test whether the power structure itself adds anything.

Source-free baselines for pruning. A strength-only curve $A((1 - d)/0.3)^\gamma$ fit on all development rows (two parameters); the median development curve, the per-density median of the development responses extended by the same power rule; and zero change. They answer whether the source state is worth reading at all.

Quantization baselines. For each bit-width, the per-bit mean and median of the development responses and zero change. The 5-bit prediction of every candidate is a fixed rule, the average of its committed 4- and 6-bit predictions, so that no 5-bit measurement enters any fit.

Distillation predictors. On the source axis, a linear model in $(\log \tilde{N}_0, \tilde{L}_{0,c}, \log \tilde{D}_0)$ (four parameters), its no- $\tilde{D}_0$ variant, and per-capability constants (mean and median of the development responses). On the pool axis, four forms fit on the 18 endpoint checkpoints of pools U75 and U600: a constant, $a + b \log T$, $a + b \log E$, and $a + b \log T + c \log E$; all four were frozen and applied to the six U225 runs (register v41).

4.3 Validation protocol and labels

Two axes are tested separately. On the source axis the intervention setting is seen and the source state is not, either through leave-one-step-out folds on the nine-state panel or through a frozen prospective on a new source. On the configuration axis the intervention setting is unseen: densities, bit-widths, or pools outside the fitted set, with or without a new source. For the new-source tests the order is fixed. Forms and standardization are committed; the dense capability vector of the new source is measured, since it is a permitted input; every candidate’s predictions are written to a versioned file and committed with the hash recorded; only then is the compressed or distilled outcome measured. Two fitting protocols are used in the last round: protocol A fits on all nine development states, protocol B only on states at step 64k or below, so that a 112k test stage is a stage extrapolation under B and a stage interpolation under A.

Each reported comparison carries a four-part origin code (defined with Tables 6–8): whether the candidate prediction was frozen before the measurement or is a leave-one-out estimate, whether the same-input baseline and the simple baseline were pre-specified with the candidate or computed afterwards, and whether all pre-specified forms are reported. A frozen prediction keeps its label whatever the outcome. When several baselines exist, the strongest per capability is shown and that choice is marked as retrospective. The only post-test selection in this paper is the per-capability reading of the four distillation forms, and it is labeled as such.

Which baseline answers which question. Whether the response is worth predicting at all: zero change and the per-capability constant. Whether the source state helps beyond the strength curve: strength-only and per-configuration constants against the source-conditioned model. Whether D_0 adds information once N_0 and L_0 are known: the full model against its no- D_0 variant on the same split. Whether a particular strength structure helps: the shared power form against A1, A2, and the continuous form on the same inputs. “Same budget” means the same admitted information; a baseline need not use all of it.

Statistics. Errors are mean absolute errors in nats per token per capability, and improvement is baseline MAE minus candidate MAE (positive means the candidate is better). Bootstrap intervals are computed only on the leave-one-out panels, resampling source states as clusters; the frozen prospectives cover one or two source states and are reported as point values with the number of independent sources. Repeated probes, checkpoints of one run, and stages of one source are never counted as independent sources.

5 Results

We first establish what compression does to the capability vector on the heterogeneous panel (§5.1), then test prediction arm by arm with one template (inputs and baselines; source axis; configuration axis; independent tests and boundaries) for pruning (§5.2), quantization (§5.3), and distillation (§5.4), and close with a cross-arm synthesis and the limitations (§5.5). Mechanism evidence, recovery training, and the re-analysis of an earlier ability-space grid are supporting material in Appendix B.

5.1 Capability responses on the heterogeneous panel

Pruning. Table 2 gives the per-capability damage at $d = 0.7$ and $d = 0.5$, the density at which math damage first exceeds one nat, and how QA ranks among the three capabilities before that cliff. Three things are visible. Magnitudes differ by two orders across series at the same density (OLMo-3-32B +0.01 against Gemma-3-27B +4.0 on math at $d = 0.7$). The ordering of capabilities is series-specific: in the three Qwen3 sizes and in OLMo-3-32B, QA is the least-damaged capability at every pre-cliff density and improves, by up to -0.79 nats at Qwen3-4B; in every Gemma-3 size, in Gemma-4-31B, and in Muse-30B, QA is the most-damaged capability at the mildest densities (Gemma-4-31B: +2.89 QA against +0.68 math at $d = 0.7$). OLMo-3-7B falls in between with differences of at most 0.15 nats. A noise

Model	ΔL_c at $d = 0.7$			math@0.5	d_{math}^*	QA least worst	min QA (d)
	math	code	QA				
Qwen3-0.6B	+0.49	+0.56	-0.26	+4.72	0.60	3/3 0/3	-0.26 (0.70)
Qwen3-1.7B	+0.24	+0.19	-0.51	+3.60	0.60	6/6 0/6	-0.90 (0.62)
Qwen3-4B	-0.00	-0.09	-0.79	+1.31	0.50	5/5 0/5	-1.96 (0.55)
Qwen3-8B [†]	+0.03	+0.10	-0.41	–	none by 0.60	4/4 0/4	-1.02 (0.60)
Qwen3-14B [†]	+0.06	+0.03	-0.45	–	none by 0.60	3/4 0/4	-0.80 (0.60)
Gemma-3-270M	+3.49	+4.04	+2.54	+11.21	0.70	0/2 1/2	+0.21 (0.90)
Gemma-3-1B	+1.31	+1.23	+0.83	+13.78	0.70	0/5 4/5	+0.19 (0.90)
Gemma-3-4B	+0.83	+1.47	+0.24	+7.00	0.60	1/3 2/3	+0.05 (0.90)
Gemma-3-12B	+0.68	+0.46	+0.54	+17.10	0.60	0/3 2/3	+0.07 (0.90)
Gemma-3-27B	+4.04	+4.69	+3.59	+15.21	0.70	1/2 1/2	+0.01 (0.80)
Gemma-4-31B	+0.68	+0.77	+2.89	+16.06	0.60	0/3 3/3	+0.22 (0.90)
Muse-30B	+0.17	+0.14	+0.20	+3.53	0.55	1/4 3/4	+0.02 (0.90)
OLMo-3-7B	+0.03	+0.03	+0.14	+0.22	0.35	4/8 3/8	-0.02 (0.55)
OLMo-3-32B	+0.01	+0.03	+0.02	+0.09	none by 0.30	7/10 0/10	-0.24 (0.30)

Table 2: Per-capability pruning damage on the heterogeneous panel (nats/token, native tokenizer units; within-model differences). d_{math}^* is the largest measured density with $\Delta L_{\text{math}} \geq 1$; models are grouped by series (Qwen3, Gemma-3, Gemma-4, Muse, OLMo-3); “math@0.5” is ΔL_{math} at $d = 0.5$; “QA least | worst” counts, over the pre-cliff densities, how often QA is the least-damaged and how often the most-damaged capability the last column is the most negative QA response and the density at which it occurs. [†]prospective additions measured at four densities only. Generated by v51_panel_tables.py from prune_losses.json.

screen sets the scale of what counts: the median $|\Delta L|$ at $d = 0.9$ is 0.013 nats, and of the 62 cells with a nominal improvement across all arms, 29 exceed twice that median; the rest are treated as zero. Cliffs occur between $d = 0.5$ and 0.7 for Qwen3 and Gemma, at 0.55 for Muse-30B, at 0.35 for OLMo-3-7B, and had not occurred by $d = 0.3$ for OLMo-3-32B, whose QA loss is -0.24 nats below dense at that density. Robustness is not monotone in size within a series: Gemma-3-12B is the most robust Gemma-3 size and 27B collapses earlier. We read all of this as heterogeneity that a predictor has to reproduce, not as an invariant.

Quantization. Appendix Table 5 covers 14 models. Int8 and int6 stay within 0.01 nats of dense throughout. Int4 damages small models (Gemma-3-270M +2.0, +2.4, +0.9) and barely touches large ones (Gemma-3-27B +0.33, +0.23, -0.67), and the QA improvement under int4 recurs in five models (Qwen3-1.7B, 8B, 14B, Gemma-3-12B, Gemma-3-27B), all clearing the noise screen. Int3 collapses 12 of the 14 models by 4.6 to 28 nats; OLMo-3-7B loses 2.8 to 4.7 nats and OLMo-3-32B only 0.4 to 1.2, so the collapse is common but not universal, and OLMo-3 is the robust series under both interventions. Within the tested bit-widths, a learned exponential $q_e e^{-kb}$ fits held-out bits better than the fixed $q_e 4^{-b}$ shape (pooled MAE 6.7 against 7.6 nats), which shows that the fixed base is wrong for this quantizer and nothing more; the squared step-size ratio of the quantizer between 4 and 5 bits is $(7/15)^2 \approx 0.22$ (Appendix A.2), and a loss ratio would follow from it only under additional error-distribution assumptions that we do not test.

Distillation. Black-box traces from two API teachers (600 per domain each) train Gemma-3 students under a full-trace and an answer-only recipe in a paired design. The answer-only recipe hurts math more than full-trace training (+0.19 to +0.22 against +0.09 to +0.12 nats relative to the initial student), while QA improves under every recipe (-0.58 to -1.44). Because the recipes differ in supervised tokens and update counts, this pairing does not isolate reasoning coverage; the controlled comparisons that hold tokens fixed are those of §5.4.

What the endpoint does and does not say. The same capability heterogeneity appears on independent secondary benchmarks for math and QA, where the primary-benchmark response predicts the secondary one better than any other capability’s response does, with bootstrap intervals excluding zero in most hold-out protocols; for code the cross-capability

predictor is as good or better, so code specificity is not established at this resolution (Appendix A.1). A lower QA loss does not by itself mean better QA behavior: in the four pruned OLMo-3-32B cells whose QA loss lies below dense, exact-match accuracy does not rise (Appendix A.1). The phrase “QA improves” in this paper refers to the loss.

5.2 Pruning arm

Inputs, candidates, baselines. Inputs are N_0 , D_0 , and the dense capability loss $L_{0,c}$ of the source; the candidates and baselines are those of §4.2. All fits use the nine Pythia development states.

Source axis (Table 3; Appendix Table 6). With the density held fixed and one pretraining stage held out, the configuration-indicator regression reaches MAE 0.253 (math), 0.292 (code), and 0.806 (QA) against 0.413, 0.516, and 0.500 for the strongest simple baseline; the improvement interval excludes zero for code only, and for QA the regression is worse than predicting zero change. Same-input comparisons say which input carries the gain: adding the dense anchor $L_{0,c}$ to $\{N_0, D_0\}$ has intervals above zero for math (+0.55, [0.29, 1.00]) and code (+0.66, [0.38, 1.13]), whereas adding D_0 to $\{N_0, L_{0,c}\}$ does not (+0.36, [−0.04, 0.60]; +0.40, [−0.01, 0.63]); on an earlier split that also held out sizes, the D_0 interval cleared zero in one pruning cell (math) out of six. On a frozen prospective at a new stage (step 96k on the 160M and 1.4B sources, between 64k and 143k) the frozen regression beat both its no- D_0 variant and the per-configuration median on all three capabilities, with the largest margin on math.

Configuration axis (Table 3; Appendix Table 7). On the development sources plus a new source (410M at 96k), pooled over the unseen densities 0.65 and 0.55, the frozen shared power form has MAE 0.371, 0.524, 0.759 against 1.37, 1.64, 1.61 for the strength-only curve: a source-blind curve cannot see that one state (160M at 143k) is several times more fragile than another. The same-input regression A2, added after the test, is better on math and code and within noise on QA, so the power structure adds nothing that the per-density regression lacks. Two frozen rounds on new sizes then set the range. For the 1B source, inside the development size range, at the interpolated densities over two stages the power form, A2, and the source-free median curve are within 0.05 nats of one another on every capability; per stage (Appendix B.7) A2 is best at 112k on each capability and the median curve is best at 32k, driven by QA. For the 6.9B source, five times the largest development size, the source-conditioned forms lose to the median curve on math and QA and tie on code: the regression extends the panel’s size trend into QA improvements of one to three nats that do not occur. At density 0.55 every source-conditioned form fails on every source (errors 0.6 to 4.5 nats) and a source-free curve is closest; the single-stage 1B@96k prospective (Appendix Table 9) shows the same two facts. Under protocol B (early stages only; Appendix Table 10) the 6.9B interpolation errors shrink to the level of the source-free curves without beating them.

Boundaries. Source-state information improves pruning prediction at seen densities and at interpolated densities, for sources inside the development size and stage range, and mostly for math and code. The gain does not require the shared power structure. It does not extend to a source five times larger, to density 0.55, or reliably to QA, whose response the current inputs do not carry.

5.3 Quantization arm

Inputs, candidates, baselines. The same inputs; the configuration-indicator regression per bit-width against its no- D_0 variant, the per-bit mean and median, and zero change.

Source axis. Pooled over the four bit-widths with a stage held out (Table 3), the regression improves on the per-bit median by +0.27 (math), +0.93 (code, interval excluding zero), and −0.52 (QA); adding D_0 has an interval above zero for code (+1.45, [0.15, 2.10]), a positive point estimate for math, and none for QA. The pooled number is dominated by int3, so the

Test	unseen	origin	math	code	QA
Source axis: seen setting, unseen source state					
prune: fixed d , 3×3 , leave-one-stage-out	stage	LFRA	0.253/0.413 _{med}	0.292/0.516 _{med}	0.806/0.500 _o
quant: fixed b , 3×3 , leave-one-stage-out	stage	LFRA	1.144/1.410 _{med}	0.626/1.553 _{med}	1.989/1.473 _{med}
prune: stage 96k (160M, 1.4B), all d	stage	PFRA	0.090/0.343 _{nD}	0.128/0.179 _{nD}	0.426/0.438 _{nD}
quant: new stage 96k (160M/410M/1.4B), $b \geq 4$	stage	P-RA	0.029/0.084 _{pbm}	0.104/0.094 _{pbm}	0.173/0.134 _{pbm}
quant: new stage 96k (160M/410M/1.4B), int3	stage	P-RA	1.488/1.934 _{pbm}	1.963/0.843 _{pbm}	2.046/1.727 _{pbm}
distill: δ_c , 3×3 LoRA, leave-one-stage-out	stage	LFRA	0.014/0.026 _{nD}	0.057/0.044 _{nD}	0.263/0.159 _c
Configuration axis: unseen setting, size, stage, or pool					
prune: d 0.65/0.55, dev sources + 410M@96k	d	PRFA	0.371/0.283 _{A2}	0.524/0.279 _{A2}	0.759/0.818 _{A2}
prune: 1B@96k, $d = 0.65$	size, stage, d	PFFA	0.020/0.009 _{A1}	0.092/0.031 _{A1}	0.466/0.380 _{A2}
prune: 1B@96k, $d = 0.55$	size, stage, d	PFFA	0.930/0.273 _{so}	1.550/0.018 _{so}	1.426/0.530 _{so}
quant: 1B@96k, int4	size, stage	PFFA	0.116/0.068 _{pbm}	0.046/0.012 _{pbm}	0.059/0.054 _{nD}
quant: 1B@96k, int3	size, stage	PFFA	0.690/1.414 _{pbm}	0.318/0.001 _{pbm}	0.182/0.446 _{pbm}
prune: 1B@32k+112k, d 0.9–0.6	size, stage	PFFA	0.215/0.169 _{so}	0.191/0.119 _{A2}	0.261/0.227 _{med}
prune: 1B@32k+112k, $d = 0.55$	size, stage, d	PFFA	1.790/0.939 _{so}	1.488/0.089 _{so}	2.225/0.393 _{so}
quant: 1B@32k+112k, $b \geq 4$ (5-bit rule incl.)	size, stage, b	PFFA	0.064/0.020 _{pbm}	0.041/0.028 _{pbm}	0.157/0.052 _{pbm}
quant: 1B@32k+112k, int3	size, stage	PFFA	0.256/1.466 _{pba}	0.362/1.537 _{pba}	1.972/1.000 _{pbm}
prune: 6.9B@32k+112k, d 0.9–0.6	size $\uparrow$, stage	PFFA	0.299/0.120 _{med}	0.160/0.144 _{A2}	0.909/0.120 _{med}
prune: 6.9B@32k+112k, $d = 0.55$	size $\uparrow$, stage, d	PFFA	0.615/0.618 _{A2}	0.547/0.535 _{A2}	4.472/0.904 _{so}
quant: 6.9B@32k+112k, $b \geq 4$	size $\uparrow$, stage, b	PFFA	0.181/0.036 _{pbm}	0.177/0.043 _{pbm}	0.428/0.034 _o
quant: 6.9B@32k+112k, int3	size $\uparrow$, stage	PFFA	3.233/2.046 _{pba}	3.229/1.805 _{pba}	5.306/1.744 _{pba}
distill: unseen pool U225 (Gemma-3-1B)	pool	PFFA	0.191/0.122 _c	0.117/0.069 _c	0.706/0.710 _{TE}

Table 3: Prediction tests, all three arms (MAE in nats/token per capability, candidate / strongest baseline, bold = better). The candidate is the source-conditioned predictor of each arm (configuration-indicator regression on the source axis and for quantization; shared power form for pruning on the configuration axis; E -only form for the distillation pool test). The baseline shown is the stronger of the same-input and the simple baseline per capability, named by code: nD = no- D_0 variant, med = per-configuration median or median development curve, 0 = zero change, so = strength-only curve, A1 = $\gamma=1$ power form, A2 = per-density regression with fixed interpolation, ct = continuous two-term form, pbm/pba = per-bit median/mean, c = constant, TE = joint $T+E$ form. “unseen” lists what the test holds out (size $\uparrow$ = five times beyond the development range). Origin code, four letters: prediction (P frozen prospective, L leave-one-out), same-input baseline, simple baseline (F pre-specified with the candidate, R computed after the test, – none), selection (A: all pre-specified forms reported). Improvements, both baselines separately, and bootstrap intervals are in Appendix Tables 6–9. Generated by v52_prediction_tables.py.

frozen step-96k test on three new sources is reported by regime: at 4 bits and above the responses are near zero and candidate and median errors are both small (0.03 against 0.08 for math, 0.10 against 0.09 for code, 0.17 against 0.13 for QA), which demonstrates no gain

at that error scale and does not show that none exists; at int3 the candidate is better for math (+0.45) and worse for code (-1.12) and QA (-0.32), so the magnitude of the collapse does not transfer consistently across capabilities.

Configuration axis (Table 3; Appendix Table 8). The tested settings are the integer bit-widths; there is no integer between 3 and 4, so the unseen-configuration tests are the 5-bit rule and the unseen sizes. For the in-range 1B source the regression is worse than the per-bit median at 4 bits and above, and at int3 it is better for math and code (+1.2 nats each) and worse for QA (-1.0). For the 6.9B source it is worse everywhere, by 0.1 to 0.4 nats above 3 bits and by 1.2 to 3.6 nats at int3, where it overshoots the collapse. Under protocol B (Appendix Table 10) the 6.9B differences above 3 bits shrink to at most 0.05 nats and at int3 the regression is ahead at 112k (1.29 against 3.8 to 4.1) and behind at 32k. The single-stage 1B@96k test had the regression ahead on the int3 aggregate (0.40 against 0.62 for the median and 2.46 without D_0) with the same math-positive, code-negative pattern per capability.

Boundaries. In the near-zero regime (4 bits and above) no predictor is distinguishable from a per-bit constant on these panels. In the collapse regime (int3) the source-conditioned regression carries information for in-range sizes, with a sign that differs by capability, and it does not transfer to a five-times larger source. Which of these statements holds depends on the fitting protocol, as the protocol B exceptions show, so a quantization result in this paper is always stated with its precision regime and protocol.

5.4 Distillation arm

Inputs, candidates, baselines. The response is the signed transfer δ_c relative to the initial student. On the source axis the inputs are the student’s N_0 , D_0 , and $L_{0,c}$ with a fixed teacher and pool; on the pool axis the inputs are the processed tokens T and the reuse count $E = T/D_U$ of a fixed student.

Source axis (Table 3). With one stage held out on the nine-state LoRA panel, the linear model in $\{N_0, L_{0,c}, D_0\}$ halves the error of a constant for math (0.014 against 0.031), ties for code (0.057 against 0.061, with the no- D_0 variant at 0.044), and is worse than a constant for QA (0.263 against 0.159). Nine cells give no usable interval; these are point estimates.

Configuration axis (Table 3; Appendix Table 8). Four forms frozen on the endpoint pools U75 and U600 were applied to an unseen middle pool U225, whose reuse count (2.3 to 4.7) lies inside the endpoint range; because the endpoints are pool prefixes and U225 is randomly sampled, the sampling protocol changes as well. The reuse-count form beats the constant only for QA (0.706 against 1.449) and ties the joint form; for math and code the constant is best (0.122, 0.069) and the fitted forms over-extrapolate. All four forms were frozen and are reported; the per-capability recommendation (reuse count for QA, constant for math and code) is a reading made after the result. The residual is dominated by systematic bias: the QA reuse-count prediction is -0.87 nats off at one budget and +0.48 at the other, a form misfit, and around that bias the variation across pool samples exceeds the variation across training seeds (QA 0.43 against 0.04 nats), which describes this design of three pools and two seeds and decomposes no population variance.

Boundaries. A transfer predictor from the student state exists for math on the controlled panel; for QA a constant is better on both axes tested; the reuse-count coordinate carries information for QA on the unseen pool with a budget-dependent bias. Both panels are small, the pool test changes two things at once, and the absolute-exposure matrix that separates them is outside the data cut-off of this paper.

5.5 Cross-arm synthesis and limitations

Which inputs help, and when. The dense capability vector of the source is the input with the clearest support: on the controlled pruning panel adding it has intervals above zero for

math and code, and every successful frozen test uses it. Training tokens D_0 add information beyond $\{N_0, L_{0,c}\}$ with interval support in one cell (quantization, code) and positive point estimates in most others; they never help QA. Source information beats source-free curves for pruning inside the development range and at seen or interpolated densities, and for quantization only in the collapse regime and for in-range sizes.

Which forms add nothing. On the same inputs, the shared power form never beats the per-density regression with a fixed interpolation rule, so no exponent is established by these data. At a new in-range size the per-density regression matches the source-free median curve on the pooled stages and beats it at the late stage only; at a five-times larger size it loses.

Which transfers were tested. Unseen stage inside the range: supported for pruning (leave-one-out and frozen). Unseen density inside the fitted range: supported for in-range sizes. Density 0.55: fails for every source-conditioned form. Unseen size inside the range: on par with source-free curves. Size five times beyond the range: fails in both arms. Unseen bit-width: only the 5-bit rule, which is worse than a per-bit constant. Unseen pool: reuse count helps QA only. Each of these is one or two source states or one pool; none is a population statement.

Resource-ratio models. A method-agnostic response in the storage or active-parameter ratio was tested on the heterogeneous panel: at the common storage ratio 0.5, pruning to $d = 0.5$ costs median +5.9, +8.2, and +2.9 nats while int8 quantization costs none; over 75 storage-matched pairs (14 with both sides pre-cliff) the mean absolute method gap is 9.3 nats with sign agreement 0.61, and the two shared-law hypotheses fit on pre-cliff cells reach held-out sign accuracies of 0.51 and 0.37. These particular models are rejected; whether another single form could unify the arms is untested.

From predictors to selection. A capability- and cost-aware method selection would use predictors like these as loss constraints. An offline replay with frozen predictions exists and quantifies the decision cost of prediction error, but no held-out agent or selection validation was run, so nothing here is a validated selector.

Limitations. The controlled panels have three sizes, three stages, and three pools, so intervals are conditional on the panel. QA is the capability the current inputs carry worst on every axis. The quantization conclusions depend on the precision regime and on the fitting protocol. The distillation pool test changes sampling protocol and pool size together. Code specificity of the endpoint was not detected at the available resolution. Recovery training, geometry, and the M1/M2 shared-law tests rest on single models or pre-cliff subsets and are reported as supporting evidence only.

6 Conclusion

We asked whether the model state observable before compression, together with the intervention setting, predicts the capability-conditioned loss after pruning, quantization, and distillation, and over which ranges such predictions transfer. The answer has three parts. Capability responses are heterogeneous across capabilities and model series, in sign as well as magnitude, so a capability-conditioned endpoint is necessary and no single ordering holds. Source-state information helps: the dense capability vector is the input that carries prediction, and source-conditioned pruning predictors beat source-free curves at seen densities and at interpolated densities on the development sources, mostly for math and code, and match them at a new in-range size; the particular power structure adds nothing over a per-density regression, and training tokens add little beyond the dense anchor. The transfer range is bounded: the same predictors fail at density 0.55 and at a source five times larger, quantization predictors add value only in the collapse regime and only under some fitting protocols, and for distillation a reuse-count coordinate helps QA on an unseen pool while a constant is best for math and code. Method-agnostic resource-ratio laws fail a matched-storage test. These results identify conditional predictive relationships and their tested limits, and they provide the measurements, registers, and comparison protocol on which

capability-conditioned scaling-down laws can be developed; extending the size range of the development panel and finding inputs that carry the QA response are the next steps, and both require new experiments.

References

- Tamay Besiroglu, Ege Erdil, Matthew Barnett, and Josh You. Chinchilla scaling: A replication attempt. arXiv preprint arXiv:2404.10102, 2024.
- Akshita Bhagia, Jiacheng Liu, Alexander Wettig, David Heineman, Oyvind Tafjord, Ananya Harsh Jha, Luca Soldaini, Noah A. Smith, Dirk Groeneveld, Pang Wei Koh, Jesse Dodge, and Hannaneh Hajishirzi. Establishing task scaling laws via compute-efficient model ladders. In Conference on Language Modeling (COLM), 2025.
- Dan Busbridge, Amitis Shidani, Floris Weers, Jason Ramapuram, Etai Littwin, and Russ Webb. Distillation scaling laws. In International Conference on Machine Learning, 2025.
- Xiaodong Chen, Yuxuan Hu, Xiaokang Zhang, Yanling Wang, Cuiping Li, Hong Chen, and Jing Zhang. P² law: Scaling law for post-training after model pruning. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics, 2025.
- Tim Dettmers and Luke Zettlemoyer. The case for 4-bit precision: k-bit inference scaling laws. In International Conference on Machine Learning, 2023.
- Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In International Conference on Machine Learning, 2023.
- Elias Frantar, Carlos Riquelme, Neil Houlsby, Dan Alistarh, and Utku Evci. Scaling laws for sparsely-connected foundation models. In International Conference on Learning Representations, 2024.
- Samir Yitzhak Gadre, Georgios Smyrnis, Vaishaal Shankar, Suchin Gururangan, Mitchell Wortsman, Rulin Shao, Jean Mercat, Alex Fang, Jeffrey Li, Sedrick Keh, Rui Xin, Marianna Nezhurina, Igor Vasiljevic, Luca Soldaini, Jenia Jitsev, Alex Dimakis, Gabriel Ilharco, Pang Wei Koh, Shuran Song, Thomas Kollar, Yair Carmon, Achal Dave, Reinhard Heckel, Niklas Muennighoff, and Ludwig Schmidt. Language models scale reliably with over-training and on downstream tasks. In International Conference on Learning Representations, 2025.
- Andrei Ghita, Nirav Desai, and Alex Boier. Scaling laws for task-specific LLM distillation. arXiv preprint arXiv:2606.24747, 2026.
- Guy Gur-Ari, Daniel A. Roberts, and Ethan Dyer. Gradient descent happens in a tiny subspace. arXiv preprint arXiv:1812.04754, 2018.
- Babak Hassibi and David G. Stork. Second order derivatives for network pruning: Optimal brain surgeon. In Advances in Neural Information Processing Systems, 1992.
- Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In International Conference on Learning Representations, 2023.
- Ajay Jaiswal, Zhe Gan, Xianzhi Du, Bowen Zhang, Zhangyang Wang, and Yinfei Yang. Compressing llms: The truth is rarely pure and never simple. In International Conference on Learning Representations, 2024.
- Tanishq Kumar, Zachary Ankner, Benjamin F. Spector, Blake Bordelon, Niklas Muennighoff, Mansheej Paul, Cengiz Pehlevan, Christopher Ré, and Aditi Raghunathan. Scaling laws for precision. In International Conference on Learning Representations, 2025.
- Yann LeCun, John S. Denker, and Sara A. Solla. Optimal brain damage. In Advances in Neural Information Processing Systems, 1989.

- Margaret Li, Sneha Kudugunta, and Luke Zettlemoyer. (mis)fitting: A survey of scaling laws. In International Conference on Learning Representations, 2025.
- Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. Quantization hurts reasoning? an empirical study on quantized reasoning models. In Conference on Language Modeling (COLM), 2025.
- Nicholas Lourie, Michael Y. Hu, and Kyunghyun Cho. Scaling laws are unreliable for downstream tasks: A reality check. In Findings of the Association for Computational Linguistics: EMNLP 2025, 2025.
- Eric J. Michaud, Ziming Liu, Uzay Girit, and Max Tegmark. The quantization model of neural scaling. In Advances in Neural Information Processing Systems, 2023.
- David Owen. How predictable is language model benchmark performance? arXiv preprint arXiv:2401.04757, 2024.
- Andrei Panferov, Alexandra Volkova, Ionut-Vlad Modoranu, Vage Egiazarian, Mher Safaryan, and Dan Alistarh. Unified scaling laws for compressed representations. In Advances in Neural Information Processing Systems, 2025.
- Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In Advances in Neural Information Processing Systems, 2024.
- Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In Advances in Neural Information Processing Systems, 2023.
- Rylan Schaeffer, Hailey Schoelkopf, Brando Miranda, Gabriel Mukobi, Varun Madan, Adam Ibrahim, Herbie Bradley, Stella Biderman, and Sanmi Koyejo. Why has predicting downstream capabilities of frontier ai models with scale remained elusive? In International Conference on Machine Learning, 2025.
- Ayan Sengupta, Siddhant Chaudhary, and Tanmoy Chakraborty. Pruning laws for large language models. arXiv preprint arXiv:2504.04342, 2025. v2, EMNLP 2026; v1 (2025) was titled Compression Laws for Large Language Models.
- Pratyusha Sharma, Jordan T. Ash, and Dipendra Misra. The truth is in there: Improving reasoning in language models with layer-selective rank reduction. In International Conference on Learning Representations, 2024.
- Ibne Farabi Shihab, Sanjeda Akter, and Anuj Sharma. Beyond variance: Knowledge-aware LLM compression via fisher-aligned subspace diagnostics. arXiv preprint arXiv:2601.07197, 2026.
- Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. A simple and effective pruning approach for large language models. In International Conference on Learning Representations, 2024.
- Lu Yin, Ajay Jaiswal, Shiwei Liu, Souvik Kundu, and Zhangyang Wang. Junk DNA hypothesis: Pruning small pre-trained weights irreversibly and monotonically impairs “difficult” downstream tasks in LLMs. In International Conference on Machine Learning, 2024.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. Galore: Memory-efficient llm training by gradient low-rank projection. In International Conference on Machine Learning, 2024.
- Chenxi Zhou, Pengfei Cao, Jiang Li, Bohan Yu, Jinyu Ye, Jun Zhao, and Kang Liu. Task-stratified knowledge scaling laws for post-training quantized large language models. In Findings of the Association for Computational Linguistics: ACL 2026, 2026.

A Appendix

A.1 Probes and the capability-conditioned loss

Each capability is measured on a fixed probe set of prompt-reference pairs drawn once, with a fixed seed, from held-out benchmark splits that are disjoint from each training and recovery source used in the paper. The primary measurement benchmarks are MATH-500 for mathematical reasoning, MBPP for code generation, and 2WikiMultihopQA for multi-hop question answering. For each capability we draw 128 probes and split them by index parity into an estimation half, used only for gradient and Fisher statistics, and a measurement half of 64 probes on which each reported loss is computed, so that mechanism statistics and evaluations do not share sampling noise. Secondary benchmarks used for the measurement checks below are GSM8K, HumanEval, and HotpotQA on the pruning panel (no training involved) and SVAMP, HumanEval, and TriviaQA where distilled models are scored, chosen to exclude the benchmarks from which the distillation traces were derived; a sample-level audit found no overlap between any measurement probe and any training pool. The measurement half is 64 probes per capability for every result in the paper except where a 512-probe re-measurement is stated.

Capability specificity as a prediction test. On a six-model, five-density pruning panel (d from 0.9 to 0.7), the response on the primary benchmark of each capability was used to predict the response on the secondary benchmark of the same capability through a two-parameter affine map fit out of fold, and compared with the best cross-capability predictor chosen the same way. With whole models held out, the same-capability gain is +0.04 nats for math (95% model-bootstrap interval [0.03, 0.05]) and +0.14 for QA ([0.02, 0.27]); with whole densities held out it is +0.14 ([0.03, 0.30]) and +0.21 ([0.02, 0.52]). For code the cross-capability predictor is as good or better (-0.06 , $[-0.16, -0.00]$ with densities held out), so code specificity is not established at this resolution. Six model clusters give descriptive intervals only.

Loss and behavior. Loss-to-accuracy links were fit on GSM8K accuracy, MBPP pass@1, and TriviaQA exact match and token F1 across seven pruned models; all four outcomes are compatible with one cross-family sigmoid in the loss, with no resolved family-specific term. A lower QA loss does not by itself imply better QA behavior: the four pruned OLMo-3-32B cells whose QA loss lies below dense show no increase in exact match. This audit covers pruning only.

Reference conditions. For a subset of models the direct-answer and given-reasoning-answer conditional distributions were measured in addition to the full-solution reference; they differ from it for math and QA and coincide for code, where the reference is the function body, so the numbers in this paper are specific to the full-solution reference and are not evidence of format robustness.

The loss is the conditional cross-entropy of the reference completion given the prompt, summed over completion tokens and divided by the number of completion tokens, as in Eq. equation 2. Prompts and references are tokenized with the model’s own tokenizer and truncated to 1024 tokens; prompt tokens are masked out of the loss. All within-model responses ΔL_c are differences in native-token nats on identical token sequences. Because native-token units differ across tokenizers, cross-family comparisons of absolute levels use per-byte normalization or are reported separately, and no cross-tokenizer quantity is treated as a common numerical scale. Negative responses are retained as measured; nothing is clipped at zero.

A.2 Compression protocols

Pruning. Global magnitude pruning removes the smallest weights across all transformer matrices (attention and MLP projections; embeddings, output head, biases, and norms are excluded) to a target retained density d . The global threshold is estimated from a seeded sample of absolute weights so that the same procedure scales to 30B-parameter models; each

density is applied to the same dense reference weights, which are held on the CPU for the largest models so that a single 80 GB device suffices.

Quantization. Weights are fake-quantized per output channel with symmetric round-to-nearest at bit-width b , using the integer range $[-(2^{b-1} - 1), 2^{b-1} - 1]$ and a per-channel scale set by the channel’s maximum absolute value; activations are left in bfloat16. The dense weights are restored between bit-widths. The quantizer is `fake_quantize_per_output_channel` in the released code: $q_{\max} = 2^{b-1} - 1$ is the largest positive integer, integers are clamped to $[-q_{\max}, q_{\max}]$ ($2^b - 1$ values), and the scale is the channel’s maximum absolute weight divided by $q_{\max}$. With a fixed clipping range the step size from 4 to 5 bits shrinks by 7/15, so the squared step ratio is $(7/15)^2 \approx 0.22$; in general the squared step ratio between adjacent bit-widths is $(q_{\max}(b)/q_{\max}(b+1))^2$, not a constant 1/4. A squared step ratio becomes a loss ratio only under assumptions about the error distribution and the local response that we do not test, so the fixed 4^{-b} shape is one candidate among others rather than a derived reference. Nominal storage ratios are computed from bit-widths; no memory or latency was measured.

Distillation. Students are fine-tuned on black-box teacher traces (a frontier API teacher answering GSM8K, HotpotQA, and CodeAlpaca prompts; 600 traces per domain) under the full-trace recipe, with strict LoRA adapters of rank 16 and scaling 32 on the query, key, value, output, gate, up, and down projections, AdamW at learning rate 10^{-4} , an effective batch of 16 sequences, and a cosine schedule with 3% warmup. A run aborts if the adapter library is unavailable and does not fall back to full fine-tuning silently, and the actual training mode and trainable parameter count are recorded in each manifest. Token accounting distinguishes processed tokens (all input tokens seen, including repetitions), unique pool tokens, and supervised completion tokens. The data pool of U examples per domain is either the first U rows of the trace file (the historical protocol) or a random sample drawn with a pool-sampling seed that is independent of the training seed; pools are identified by the sorted hashes of their samples and pairwise overlaps are reported. The historical schedule spanned a fixed number of epochs, so its cosine horizon varied with pool size; the controlled matrix of §4.2 uses a schedule defined on the same absolute processed-token budget for each pool, and the two protocols are kept apart in fitting.

A.3 The controlled Pythia panel

The panel uses the public Pythia checkpoints at sizes 160M, 410M, and 1.4B and pretraining steps 16,000, 64,000, and 143,000, which correspond to 33.6B, 134.2B, and 299.9B processed tokens at the published batch of 1024×2048 tokens per step. The size coordinate N_0 counts transformer-matrix parameters only (the query, key, value, output, and MLP matrices of each layer), computed from the architecture configuration, and the training-tokens coordinate is $D_0 = \text{step} \times 2,097,152$. Each checkpoint’s weight identity was verified before use: the 2.8B revisions published for different steps were found to load identical weights and were excluded, and an integrity guard now rejects any pair of nominally different checkpoints with identical measured panels. New sources added for the prospective tests (Pythia-1B at steps 32,000, 96,000, and 112,000; Pythia-6.9B at 32,000 and 112,000) were confirmed to have not entered any fit or model selection before their predictions were frozen.

A.4 Registration, freezing, and labels

Each prediction test in Table ?? follows one order. The functional forms, the development set, and each standardization and coefficient are fixed and committed first. For a new source the dense capability loss is then measured, since it is a permitted pre-compression input; the predictions for each configuration and candidate are written to a versioned file and committed to the repository, with the commit hash recorded; and only then is the compressed or distilled outcome measured. Information budgets are reported separately: K0 uses pre-compression information only, K1 adds one pre-specified target-side calibration point, and post-hoc rescaling on target outcomes is reported only as an oracle diagnostic. Each comparison carries the four-part origin code defined with

Tables 6–8; the label of a frozen prediction never depends on its outcome. Register files (paths in the repository; content hashes in docs/MANUSCRIPT_REWRITE_MAP.md): v36b-input-comparison/summary.json (leave-one-step-out source axis, both arms), v38-prospective/register.json (frozen step-96k coefficients), v40-prune-strength/register.json (shared power form and source-free curves), v41-distill-newpool/summary.json (the four frozen pool forms), v46-p1-newsource/predictions_frozen.json (1B@96k), v49-p1v2/register.json and predictions_*.json (1B and 6.9B pairs, protocols A and B). When a diagnostic overturns an earlier headline, the earlier claim is downgraded in the text and the prediction itself stays as committed.

A.5 Reproduction

All tables are generated from machine-readable result files by CPU-only scripts that run in seconds and require neither model weights nor a GPU; the repository mirrors the scripts and the result files they read, and the main prediction table is produced by a single command. Model measurements are reproducible from the recorded revisions, probe seeds, and protocols above.

B Supporting analyses

B.1 Capabilities occupy distinct regions of gradient space

Convergent validity (block structure). For each of nine benchmarks we measure a gradient signature (the benchmark-conditional diagonal Fisher) and compare all pairs. After removing the component shared across benchmarks, signatures of same-capability benchmarks are positively aligned (cosine +0.36 to +0.45) while signatures of different capabilities are anti-aligned (−0.23 to −0.25), in each of the ten models from five series for which the block analysis was run (Figs. 1–10); the C4 control sits between blocks. (preliminary evidence: point estimates; example-bootstrap, permutation, and prompt-stability tests are left to future work) The mathematics benchmarks cluster regardless of format, and within-block fine structure matches intuition (SVAMP, grade-school word problems, is the loosest member of the math block). A projected-gradient SVD version gives the same verdict in subspace terms: principal-angle similarity within capability exceeds cross-capability similarity by 2–4×.

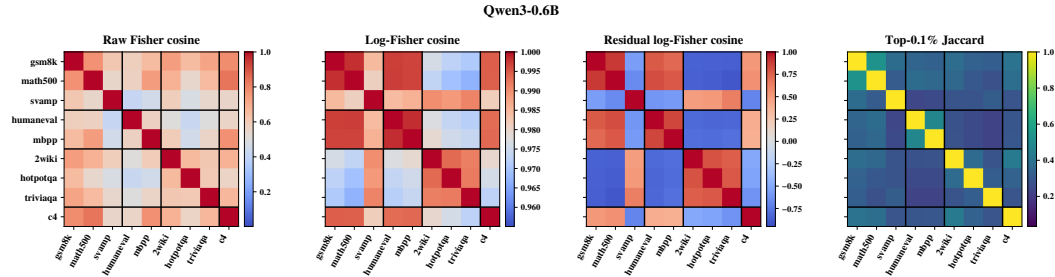


Figure 1: Capability regions in gradient space for Qwen3-0.6B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

Selective ablation (preliminary evidence: single model; random, magnitude-matched, and global-Fisher controls are left to future work). A first targeted test: select the top 0.05% of weight coordinates by each capability’s residual Fisher signature (excluding coordinates shared with other capabilities) and zero them. The damage matrix is nearly diagonal (Gemma-3-1B; rows ablate, columns measure, nats/token):

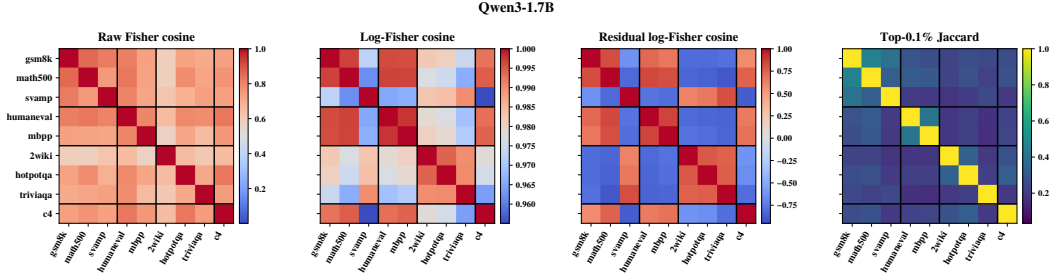


Figure 2: Capability regions in gradient space for Qwen3-1.7B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

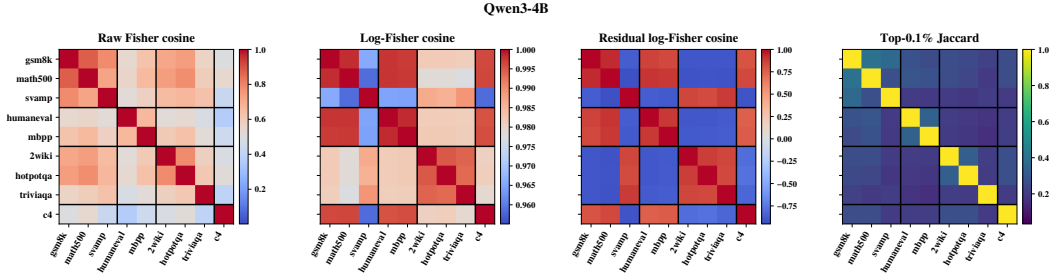


Figure 3: Capability regions in gradient space for Qwen3-4B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

Ablated region	math	code	QA
math	+4.63	+0.43	+0.05
code	+0.42	+7.14	+0.00
QA	+0.01	+0.00	+2.29

Zeroing 0.05% of weights, selected from gradients alone, selectively removes one capability while leaving the others nearly untouched (the QA ablation moves code by +0.0001). Pending multi-model replication and the control ablations above, we describe the regions as causally implicated and partially separable; we do not call them proven causal carriers; if confirmed, ablation itself is a form of targeted scaling-down.

B.2 The pruning law

The mechanism form below fits the entire pre-cliff grid. With the measured alignment term subtracted, the curvature residual follows a power law in the deleted capability mass $m_c(d)$:

$$\Delta L_c(d) = \underbrace{\bar{g}_c^\top \delta w_d}_{\text{measured}} + B \left[\underbrace{m_c(d)}_{\text{measured}} \right]^\gamma, \quad (3)$$

with in-sample $R^2 = 0.82\text{--}0.996$ over 24 capability $\times$ model fits. The held-out battery (1000-resample cell bootstrap) is where the form is tested: in leave-one-family-out transfer the shared- γ mechanism form attains MAE 2.17 nats versus 10.3 for per-capability raw sparsity and 8.2 for the direct-damage baseline (both CIs exclude zero), so the deleted-mass coordinate is what transfers across families; it also beats the direct-damage baseline in fit-shallow/predict-deep (3.11 vs. 4.24) and raw sparsity in leave-largest-model-out (1.41 vs. 2.15), while its edge over raw sparsity within-model does not reach significance (without the

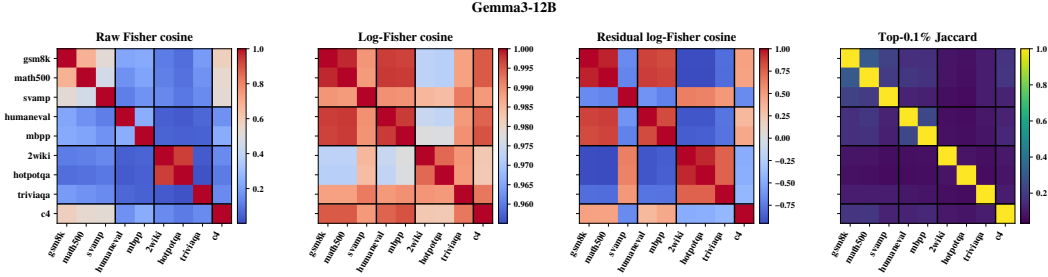


Figure 4: Capability regions in gradient space for Gemma3-12B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

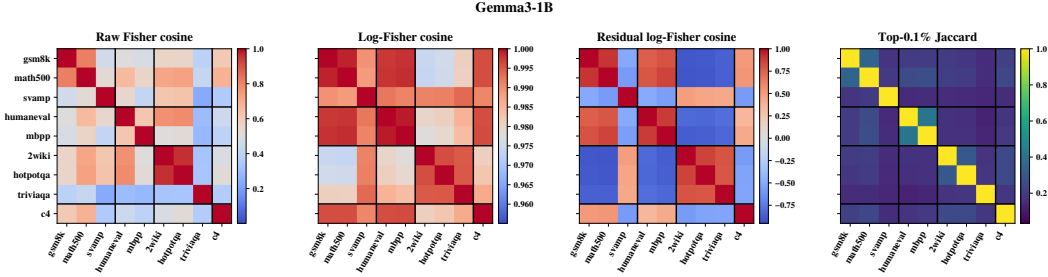


Figure 5: Capability regions in gradient space for Gemma3-1B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

first-order term, QA is unfittable, $R^2 \leq 0.67$; with it, QA joins the law). Two structural results: (i) in the deleted-mass coordinate the exponent concentrates at $\gamma \in [1.2, 2.2]$ across all families, versus 3.2–8.7 in the raw $(1-d)$ coordinate, which is what makes the form portable; and (ii) within a model, γ is shared across capabilities (e.g. Qwen3-0.6B: 1.41/1.56/1.43; Gemma3-4B: 1.66/1.71): capability differences appear to be carried by the two measured quantities, and the fitted structure is two parameters per model. This mechanism form uses gradient information of the dense model and is therefore not a K0 predictor; it is reported as an upper comparator for the reduced forms of the main text, not as a bound.

B.3 The signed first-order term predicts, in both directions

The alignment term $\bar{g}_c^\top \delta w$ is the family coefficient that carries sign. Its sign predicts the presence or absence of the capability-selective improvement in each case measured: the QA term of Qwen3 is negative and QA improves under pruning (predicted -0.83 vs. measured -0.79 at 4B, $d = 0.7$); the QA term of Gemma is positive and no improvement exists at any size; the term for OLMo-3-7B is positive (mild degradation, prediction $+0.015$ = measurement at $d = 0.8$, code) while the term for OLMo-3-32B is negative, and deep-pruning QA improvement duly appears. At the level of individual model $\times$ capability $\times$ density cells in the mild regime, sign prediction accuracy is 67.6% (Wilson 95% CI $[58.3, 75.7]$, $n=108$), above chance, with the family-level pattern (Qwen and OLMo-32B QA improvements, Gemma none, an OLMo size flip) called in advance. Beyond the mild regime, first- plus second-order predictions fail by orders of magnitude at the cliff in each family: the collapse is non-perturbative in loss space, confirming the two-regime structure of equation 3.

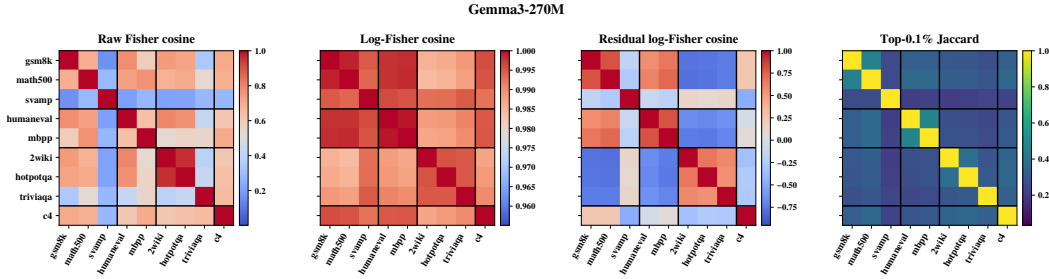


Figure 6: Capability regions in gradient space for Gemma3-270M: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

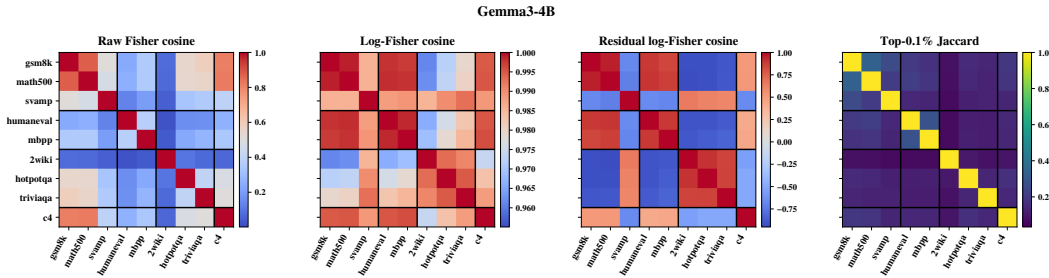


Figure 7: Capability regions in gradient space for Gemma3-4B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

B.4 The cliff is a geometric threshold crossing

Measuring the capability geometry of the pruned models accounts for the location of the cliff on the three models tested. The capability-region mass retention (the fraction of the capability-exclusive top coordinates of the dense model still prominent after pruning) declines smoothly from the first densities (no cliff in geometry) while loss is still flat; the behavioral cliff occurs where retention crosses a critical value. The critical value is family-specific (Gemma: 0.035–0.10 across 1B/4B and all capabilities; Qwen: 0.13–0.23) but consistent within a family across sizes and capabilities: the cliff boundary $d_c^*(N_0, f)$ reduces to one critical-retention constant per family, turning the least elegant fitted parameter of the two-regime law into a measurable, smoothly evolving order parameter. (preliminary evidence: three models, two families; a multi-model extension is left to future work)

B.5 Recovery arm: redeemable damage and the fitted plateau

Recovery training on general text fits the recovery law cleanly (Gemma3-270M pruned to $d = 0.6$; $R^2 \approx 1.0$): $\beta'_{\text{math}} = 1.23$, $\beta'_{\text{code}} = 1.02$, with fitted plateaus under the fixed LoRA recipe and the largest measured budget of 16M tokens, $L_c^{\text{plateau}} - L_c^{\text{dense}}$, of 1.21 (math) and 1.93 (code) nats; about 80% of this pruning damage is redeemed within 8M tokens and the remainder persists up to the largest budget measured under this recipe, while QA recovers completely. The plateau is a fitted quantity at 16M tokens, not an established asymptote, and the saturating form does not describe the non-monotone aligned-data trajectory below. (preliminary evidence: one model, one density, one recipe; with three-point ladders the largest-budget extrapolation of the saturating form beats a carry-forward baseline in only 4 of 9 fits, so the ladder is too sparse to claim extrapolation, and a denser multi-model grid is left to future work) Two mechanism results: recovery on aligned data (teacher traces) heals

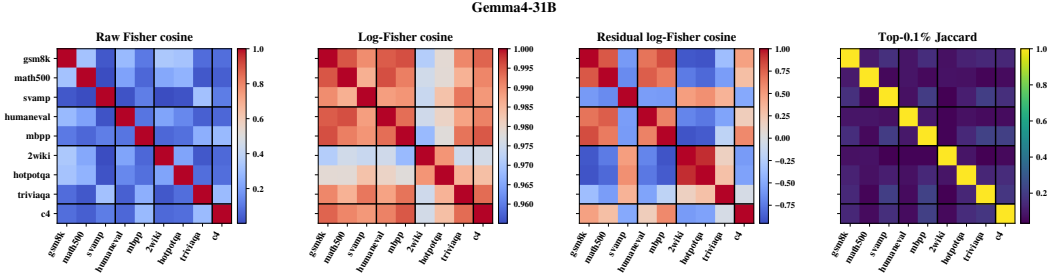


Figure 8: Capability regions in gradient space for Gemma4-31B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

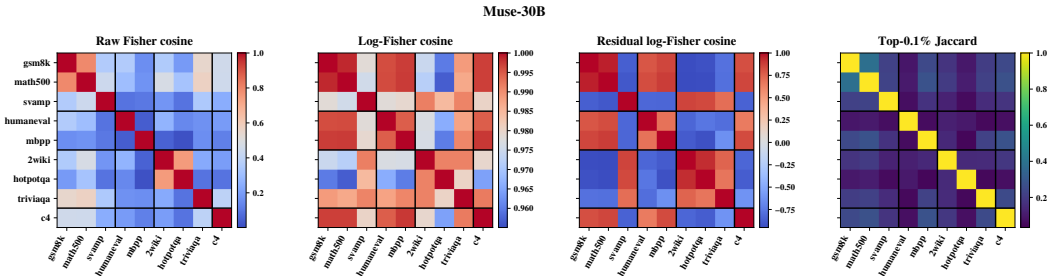


Figure 9: Capability regions in gradient space for Muse-30B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

substantially faster at small budgets (math 2.76 vs. 3.47 at 0.5M tokens), as the gradient-overlap prediction requires, but over-training on narrow aligned data re-damages the model (QA 4.95 $\rightarrow$ 9.75 from 0.5M to 8M tokens), while general-text recovery is monotone. Aligned recovery is fast at small budgets and harmful when over-trained on narrow data; general recovery is slow and monotone on this model. Quantization damage past the bit cliff is markedly less redeemable under the same recipe (residuals $\approx$ 2.6–2.8 after int3) than pruning damage at matched severity.

B.6 Scale and family structure

Within-family robustness is non-monotonic in two families independently. Gemma-3 robustness peaks at 12B (27B collapses far earlier, +4.0 vs. +0.68 at $d = 0.7$), mirroring the prior ability-space observation of a Qwen3 14B peak. A mid-scale robustness peak is a reproducible pattern of scaling-down.

A candidate predictor, honestly retired. The top-concentration statistic κ_c tracked robustness within the Qwen3 ladder and was pre-registered as a predictor; the full grid retires it (cross-family inversion: OLMo, the most robust family, has the lowest concentration; within-Gemma top-end violation).

Matched-size panel ($\sim$ 30B). At matched scale the family coefficient dominates: ΔL_{math} at $d = 0.7$ spans +0.02 (OLMo-3-32B), +0.17 (Muse-30B), +0.68 (Gemma4-31B), +4.04 (Gemma3-27B). OLMo-3-32B survives 70% pruning with $\leq +0.63$ nats on each capability. Family identity is a first-order determinant of prunability on this panel; which training-data or recipe properties carry it was not measured.

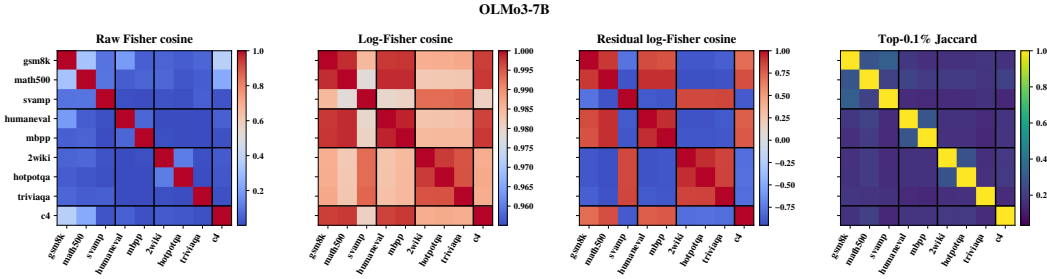


Figure 10: Capability regions in gradient space for OLMo3-7B: pairwise similarity of per-benchmark gradient signatures under four metrics (raw Fisher cosine, log-Fisher cosine, shared-component-removed log-Fisher cosine, top-0.1% coordinate Jaccard). Black lines separate the math, code, QA, and control blocks.

B.7 New-source perspectives, second round

The second frozen round (Table 10) added seven pruning densities (0.9, 0.8, 0.75, 0.7, 0.65 and 0.6 as interpolation, 0.55 as extrapolation) and five bit-widths (8, 6, 5, 4, 3) on Pythia-1B at steps 32k and 112k. Two fitting protocols were frozen at the same time. Protocol A fits each candidate on the full nine-state development panel. Protocol B fits on development states at step 64k or below, so that its 112k rows test a training-stage extrapolation on top of the size change. The 5-bit prediction of the class-indicator candidate is a fixed rule, the average of its committed 4- and 6-bit predictions, chosen so that no 5-bit measurement enters any fit. Candidate names follow §4.2: power is the shared power form, A2 the per-density source regression with a fixed interpolation rule, A1 its $\gamma = 1$ variant, cont a continuous two-term polynomial in $1 - d$, and the three source-free baselines are strength-only, the median development curve, and zero change. For quantization, full uses $\{N_0, L_0, D_0\}$, no-D0 drops the training-stage term, and mean and median are per-bit constants from the development panel.

Per capability, at 112k under protocol A the source regression reaches MAE 0.111 (math), 0.091 (code) and 0.135 (QA); the strongest source-free alternative is strength-only for math (0.117) and the median curve for code (0.107) and QA (0.261). At 32k the QA error of the source regression rises to 0.339 while the median curve stays at 0.193, which accounts for most of the aggregate gap. Under protocol B at 112k the power and A2 forms are ahead of strength-only at $d = 0.55$ (0.96 and 1.00 against 1.33), but with errors near one nat the ordering carries little practical value. The pair cost 22 A100 minutes.

The 6.9B pair, a size extrapolation of about five times, was measured under the same freeze (36 A100 minutes, reference model on CPU). Its dense anchors are math 1.27, code 1.30 and QA 5.11 at 32k, and 1.11, 1.21 and 5.09 at 112k. Per capability under protocol A at 112k, the source regression reaches 0.23 (math), 0.19 (code) and 0.96 (QA) against 0.05, 0.07 and 0.15 for the median curve. Under protocol B the source forms lead on code only (A2 0.058 and 0.083 at the two stages against 0.208 and 0.074 for the median curve) and trail on QA (0.32 and 0.20 against 0.08 and 0.15). At $d = 0.55$ the source-free curves win under both protocols, with source-conditioned errors between 0.55 and 2.2 nats. For quantization under protocol A the class-indicator candidate is worst or near worst above 3 bits (0.25 and 0.27 against 0.03 and 0.04 for the per-bit median) and overshoots by about four nats at 3 bits. Protocol B compresses the differences above 3 bits to the 0.02 to 0.05 range and leaves the candidate ahead at 3 bits only for the late stage (1.29 against 3.8 to 4.1).

B.8 Re-analysis of the prior ability-space grid

A prior internal study fit compression laws directly on Rasch abilities from ~ 100 -item panels over a 774-cell Qwen3 grid and reported that no derived law had held-out predictive power. Our re-analysis attributes much of this to the measurement layer: (1) the $r=1$ falsification anchor was driven almost entirely by the answer-only recipe (stratified, the anchor holds

and held-out error drops from $9.0\times$ to $1.8\times$ the noise floor); (2) same-benchmark evaluation inflated QA gains by roughly one third; (3) discrimination heterogeneity voids interval-scale ability while rankings survive ($\rho \geq 0.98$); (4) item-level damage is smooth pre-cliff, ruling out discrete skill deletion at the item level. These findings fixed the protocol of §3.

Cohort	Sources	Interventions	Probes	Role
Heterogeneous panel	12 dense checkpoints, 5 series from 4 families: Qwen3 0.6B/1.7B/4B, Gemma-3 270M–27B, Gemma-4-31B, Muse-30B, OLMo-3 7B/32B (+2 prospective: Qwen3 8B/14B)	global magnitude pruning, 10 densities in [0.3, 0.9]; RTN int8/6/5/4/3	64 per capability, primary benchmarks	ca-phenomena; cross-family transfer; noise screen
Controlled Pythia panel	sizes 160M/410M/1.4B × steps 16k/64k/143k (9 states); new sources 160M, 410M, 1.4B at 96k; 1B at 32k/96k/112k; 6.9B at 32k/112k	pruning: 4 dev densities 0.9–0.6, tests at 0.75/0.65/0.55; bits 8/6/4/3, test 5	same	development set; source-axis and configuration-axis tests
Distillation, controlled	the 9 Pythia states as students; fixed teacher pool (600 traces per domain); LoRA	fixed recipe	same	δ_c source transfer
Distillation, pools	Gemma-3-1B student; pools U75 and U600 (fit), U225 (test); 3 pool seeds × 2 training seeds	processed tokens T ; reuse count $E = T/D_U$	same	configuration axis (pool)
Distillation, heterogeneous	Gemma-3 270M–12B, OLMo-3-7B, Qwen3 0.6B–4B students; two API teachers; full-trace vs answer-only	recipe	same	phenomena only

Table 4: Cohorts and their roles. Counts are generated from the result files (v51_panel_tables.py); the two Qwen3 prospectives were measured at four densities only. The 2.8B Pythia checkpoints were excluded because the step revisions we downloaded loaded identical weights (hash check); nothing is claimed about other revisions.

Model	Series	ΔL_c at int4			ΔL_c at int3			bits
		math	code	QA	math	code	QA	
Qwen3-0.6B	Qwen3	+0.56	+0.73	+0.21	+8.83	+10.83	+6.61	8,6,5,4,3
Qwen3-1.7B	Qwen3	+0.40	+0.16	-0.21	+9.53	+9.46	+8.10	8,6,5,4,3
Qwen3-4B	Qwen3	+0.13	+0.19	+0.39	+10.73	+11.85	+5.99	8,6,5,4,3
Qwen3-8B [†]	Qwen3	+0.28	+0.48	-0.80	+12.86	+13.55	+9.45	8,6,5,4,3
Qwen3-14B [†]	Qwen3	+0.09	+0.31	-0.23	+9.30	+10.79	+6.45	8,6,4,3
Gemma-3-270M	Gemma-3	+1.99	+2.35	+0.88	+26.15	+27.94	+25.93	8,6,5,4,3
Gemma-3-1B	Gemma-3	+0.95	+0.59	+0.88	+15.33	+14.12	+9.80	8,6,5,4,3
Gemma-3-4B	Gemma-3	+0.72	+0.50	-0.27	+21.73	+19.24	+15.30	8,6,5,4,3
Gemma-3-12B	Gemma-3	+0.40	+0.32	-0.42	+20.19	+22.92	+20.75	8,6,5,4,3
Gemma-3-27B	Gemma-3	+0.33	+0.23	-0.67	+18.46	+17.15	+16.10	8,6,5,4,3
Gemma-4-31B	Gemma-4	+0.94	+0.58	+1.28	+20.35	+19.85	+14.90	8,6,5,4,3
Muse-30B	Muse	+1.58	+2.06	-0.11	+11.14	+11.18	+8.36	8,6,5,4,3
OLMo-3-7B	OLMo-3	+0.16	+0.21	-0.07	+4.70	+4.63	+2.76	8,6,5,4,3
OLMo-3-32B	OLMo-3	+0.04	+0.06	+0.02	+0.68	+1.22	+0.38	8,6,5,4,3

Table 5: Per-capability quantization damage (per-output-channel symmetric RTN, weights only; nats/token). int8 and int6 are within 0.01 nats of dense for every listed model and are omitted. [†]prospective additions. Generated by v51_panel_tables.py from quant_losses.json.

Test; unseen axis; [origin]	Line	math	code	QA
pruning: source transfer at fixed d : 3×3 Pythia panel, leave-one-step-out held-out pretraining stage (in-range) [L · F · R · A]	cand.: config-indicator OLS	0.253 (+0.16)	0.292 (+0.22)*	0.806 (-0.31)
	same-input: no- D_0	0.613	0.693	0.774
	simple: per-config	0.413	0.516	0.500
	median; zero change			
quantization: source transfer at fixed b : 3×3 Pythia panel, leave-one-step-out held-out pretraining stage (in-range) [L · F · R · A]	cand.: config-indicator OLS	1.144 (+0.27)	0.626 (+0.93)*	1.989 (-0.52)
	same-input: no- D_0	1.989	2.077	1.964
	simple: per-config	1.410	1.553	1.473
	median			
pruning: new stage 96k on 160M and 1.4B, all measured d stage 96k (interpolation between 64k and 143k) [P · F · R · A]	cand.: config-indicator OLS	0.090 (+0.25)	0.128 (+0.05)	0.426 (+0.01)
	same-input: no- D_0	0.343	0.179	0.438
	simple: per-config	0.367	0.438	0.832
	median			
quantization: new stage 96k on 160M, 410M, 1.4B; $b \geq 4$ (near-zero regime) stage 96k (interpolation) [P · - · R · A]	cand.: config-indicator OLS	0.029 (+0.06)	0.104 (-0.01)	0.173 (-0.04)
	same-input: none available	-	-	-
	simple: per-bit	0.084	0.094	0.134
	median			
quantization: new stage 96k on 160M, 410M, 1.4B; int3 (collapse regime) stage 96k (interpolation) [P · - · R · A]	cand.: config-indicator OLS	1.488 (+0.45)	1.963 (-1.12)	2.046 (-0.32)
	same-input: none available	-	-	-
	simple: per-bit	1.934	0.843	1.727
	median			
distillation: δ_c source transfer: 3×3 LoRA panel, leave-one-step-out held-out pretraining stage (in-range) [L · F · R · A]	cand.: linear { N_0, L_0, D_0 }	0.014 (+0.01)	0.057 (-0.01)	0.263 (-0.10)
	same-input: no- D_0	0.026	0.044	0.265
	simple: constant (mean); constant (median)	0.031	0.061	0.159

Table 6: Source axis: does the pre-compression source state predict the response at a fixed intervention setting? MAE in nats/token per capability; bold = smallest in the group; the number in parentheses is the improvement of the candidate over the strongest listed baseline (positive = candidate better); * = bootstrap 95% interval of that improvement excludes zero (computed only for the leave-one-out panels). Origin code: prediction origin (P frozen prospective, L leave-one-out) · same-input baseline origin · simple baseline origin (F pre-specified with the candidate, R computed after the test, - none) · selection (A: all pre-specified forms reported). Where several same-input or simple baselines exist, the strongest per capability is shown and named; that choice is retrospective. Config-indicator OLS uses $\{N_0, L_0, D_0\}$ with one coefficient set per configuration. Generated by v52_prediction_tables.py.

Test; unseen axis; [origin]	Line	math	code	QA
pruning: unseen d 0.65 and 0.55, dev sources + 410M@96k densities 0.65 (interp.) and 0.55 (extrap.) pooled [P · R · F · A]	cand.: shared power form	0.371 (-0.09)	0.524 (-0.24)	0.759 (+0.06)
	same-input: A2 (per- d regr.)	0.283	0.279	0.818
	simple: strength-only	1.369	1.635	1.608
pruning: new source 1B@32k+112k, d 0.9–0.6 (interp.) in-range size; stages 32k, 112k [P · F · F · A]	cand.: shared power form	0.215 (-0.05)	0.191 (-0.07)	0.261 (-0.03)
	same-input: A2 (per- d regr.)	0.205	0.119	0.237
	simple: median curve; strength-only	0.169	0.128	0.227
pruning: new source 1B@32k+112k, $d = 0.55$ (extrap.) in-range size; stages 32k, 112k [P · F · F · A]	cand.: shared power form	1.790 (-0.85)	1.488 (-1.40)	2.225 (-1.83)
	same-input: A1 ($\gamma=1$); A2 (per- d regr.)	1.896	1.386	1.744
	simple: strength-only	0.939	0.089	0.393
pruning: new source 6.9B@32k+112k, d 0.9–0.6 (interp.) size $5\times$ beyond dev range; stages 32k, 112k [P · F · F · A]	cand.: shared power form	0.299 (-0.18)	0.160 (-0.02)	0.909 (-0.79)
	same-input: A2 (per- d regr.)	0.292	0.144	0.982
	simple: median curve	0.120	0.144	0.120
pruning: new source 6.9B@32k+112k, $d = 0.55$ (extrap.) size $5\times$ beyond dev range; stages 32k, 112k [P · F · F · A]	cand.: shared power form	0.615 (+0.00)	0.547 (-0.01)	4.472 (-3.57)
	same-input: A1 ($\gamma=1$); A2 (per- d regr.)	0.618	0.535	2.850
	simple: median curve; strength-only	0.703	0.712	0.904

Table 7: Configuration axis, pruning: does a relation fit at seen densities and sources predict unseen densities, sizes and stages? Protocol A (full development panel) for the two P1-v2 pairs; protocol B in Appendix Table 10. MAE in nats/token per capability; bold = smallest in the group; the number in parentheses is the improvement of the candidate over the strongest listed baseline (positive = candidate better); * = bootstrap 95% interval of that improvement excludes zero (computed only for the leave-one-out panels). Origin code: prediction origin (P frozen prospective, L leave-one-out) · same-input baseline origin · simple baseline origin (F pre-specified with the candidate, R computed after the test, – none) · selection (A: all pre-specified forms reported). Where several same-input or simple baselines exist, the strongest per capability is shown and named; that choice is retrospective. Config-indicator OLS uses $\{N_0, L_0, D_0\}$ with one coefficient set per configuration. Generated by v52_prediction_tables.py.

Test; unseen axis; [origin]	Line	math	code	QA
quantization: new source 1B@32k+112k, $b \geq 4$ (incl. 5-bit rule) in-range size; stages 32k, 112k [P · F · F · A]	cand.: config-indicator OLS	0.064 (-0.04)	0.041 (-0.01)	0.157 (-0.10)
	same-input: no- D_0	0.118	0.156	0.153
	simple: per-bit median	0.020	0.028	0.052
quantization: new source 1B@32k+112k, int3 in-range size; stages 32k, 112k [P · F · F · A]	cand.: config-indicator OLS	0.256 (+1.21)	0.362 (+1.18)	1.972 (-0.97)
	same-input: no- D_0	3.218	3.565	2.897
	simple: per-bit mean; per-bit median	1.466	1.537	1.000
quantization: new source 6.9B@32k+112k, $b \geq 4$ (incl. 5-bit rule) size $5\times$ beyond dev range; stages 32k, 112k [P · F · F · A]	cand.: config-indicator OLS	0.181 (-0.14)	0.177 (-0.13)	0.428 (-0.39)
	same-input: no- D_0	0.169	0.154	0.426
	simple: per-bit median; zero change	0.036	0.043	0.034
quantization: new source 6.9B@32k+112k, int3 size $5\times$ beyond dev range; stages 32k, 112k [P · F · F · A]	cand.: config-indicator OLS	3.233 (-1.19)	3.229 (-1.42)	5.306 (-3.56)
	same-input: no- D_0	4.199	3.176	5.605
	simple: per-bit mean	2.046	1.805	1.744
distillation: unseen pool U225 from endpoint pools U75/U600 (Gemma-3-1B) pool (reuse count in-range; sampling protocol differs) [P · F · F · A]	cand.: E -only (reuse count)	0.191 (-0.07)	0.117 (-0.05)	0.706 (+0.00)
	same-input: $T+E$ joint	0.215	0.145	0.710
	simple: constant	0.122	0.069	1.449

Table 8: Configuration axis, quantization and distillation: unseen sizes and stages at seen bit-widths (protocol A; the 5-bit prediction is a fixed interpolation rule), and an unseen distillation pool. MAE in nats/token per capability; bold = smallest in the group; the number in parentheses is the improvement of the candidate over the strongest listed baseline (positive = candidate better); * = bootstrap 95% interval of that improvement excludes zero (computed only for the leave-one-out panels). Origin code: prediction origin (P frozen prospective, L leave-one-out) · same-input baseline origin · simple baseline origin (F pre-specified with the candidate, R computed after the test, – none) · selection (A: all pre-specified forms reported). Where several same-input or simple baselines exist, the strongest per capability is shown and named; that choice is retrospective. Config-indicator OLS uses $\{N_0, L_0, D_0\}$ with one coefficient set per configuration. Generated by v52_prediction_tables.py.

Test; unseen axis; [origin]	Line	math	code	QA
pruning: unseen d 0.65 and 0.55, dev sources + 410M@96k densities 0.65 (interp.) and 0.55 (extrap.) pooled [P · R · F · A]	cand.: shared power form	0.371 (-0.09)	0.524 (-0.24)	0.759 (+0.06)
	same-input: A2 (per- d regr.)	0.283	0.279	0.818
	simple: strength-only	1.369	1.635	1.608
pruning: new source 1B@96k, $d = 0.65$ (interp.) in-range size; stage 96k [P · F · F · A]	cand.: shared power form	0.020 (-0.01)	0.092 (-0.06)	0.466 (-0.09)
	same-input: A1 ($\gamma=1$); A2 (per- d regr.)	0.009	0.031	0.380
	simple: median curve	0.147	0.076	0.569
pruning: new source 1B@96k, $d = 0.55$ (extrap.) in-range size; stage 96k [P · F · F · A]	cand.: shared power form	0.930 (-0.66)	1.550 (-1.53)	1.426 (-0.90)
	same-input: A2 (per- d regr.)	1.051	1.358	1.009
	simple: strength-only	0.273	0.018	0.530
quantization: new source 1B@96k, int4 in-range size; stage 96k; seen bit [P · F · F · A]	cand.: config-indicator OLS	0.116 (-0.05)	0.046 (-0.03)	0.059 (-0.00)
	same-input: no- D_0	0.184	0.394	0.054
	simple: per-bit median	0.068	0.012	0.253
quantization: new source 1B@96k, int3 in-range size; stage 96k; seen bit [P · F · F · A]	cand.: config-indicator OLS	0.690 (+0.72)	0.318 (-0.32)	0.182 (+0.26)
	same-input: no- D_0	2.324	3.423	1.635
	simple: per-bit median	1.414	0.001	0.446
pruning: new source 1B@32k+112k, d 0.9–0.6 (interp.) in-range size; stages 32k, 112k [P · F · F · A]	cand.: shared power form	0.215 (-0.05)	0.191 (-0.07)	0.261 (-0.03)
	same-input: A2 (per- d regr.)	0.205	0.119	0.237
	simple: median curve; strength-only	0.169	0.128	0.227
pruning: new source 1B@32k+112k, $d = 0.55$ (extrap.) in-range size; stages 32k, 112k [P · F · F · A]	cand.: shared power form	1.790 (-0.85)	1.488 (-1.40)	2.225 (-1.83)
	same-input: A1 ($\gamma=1$); A2 (per- d regr.)	1.896	1.386	1.744
	simple: strength-only	0.939	0.089	0.393
quantization: new source 1B@32k+112k, $b \geq 4$ (incl. 5-bit rule) in-range size; stages 32k, 112k [P · F · F · A]	cand.: config-indicator OLS	0.064 (-0.04)	0.041 (-0.01)	0.157 (-0.10)
	same-input: no- D_0	0.118	0.156	0.153
	simple: per-bit median	0.020	0.028	0.052
quantization: new source 1B@32k+112k, int3	cand.: config-indicator OLS	0.256 (+1.21)	0.362 (+1.18)	1.972 (-0.97)

Source	Prot.	Regime	power	A2	A1	cont	str.-only	median	zero	best
1b@32000	A	prune interp d0.9-0.6	0.254	0.261	0.393	0.275	0.363	0.204	0.412	median-curve
1b@32000	A	prune extrap d0.55	2.272	2.130	2.394	2.221	0.637	1.509	2.434	strength-only
1b@32000	B	prune interp d0.9-0.6	0.249	0.218	0.321	0.239	0.236	0.224	0.412	A2
1b@32000	B	prune extrap d0.55	1.772	1.777	2.151	1.863	1.531	1.645	2.434	strength-only
1b@112000	A	prune interp d0.9-0.6	0.191	0.112	0.300	0.175	0.365	0.173	0.400	A2
1b@112000	A	prune extrap d0.55	1.398	1.235	1.838	1.356	0.311	1.311	2.237	strength-only
1b@112000	B	prune interp d0.9-0.6	0.223	0.177	0.355	0.235	0.219	0.195	0.400	A2
1b@112000	B	prune extrap d0.55	0.957	0.995	1.685	1.127	1.333	1.447	2.237	power
6.9b@32000	A	prune interp d0.9-0.6	0.476	0.482	0.608	0.501	0.483	0.165	0.160	zero
6.9b@32000	A	prune extrap d0.55	1.635	1.423	0.939	1.457	1.493	0.275	0.889	median-curve
6.9b@32000	B	prune interp d0.9-0.6	0.160	0.166	0.249	0.187	0.163	0.136	0.160	median-curve
6.9b@32000	B	prune extrap d0.55	0.705	0.549	0.677	0.630	0.252	0.148	0.889	median-curve
6.9b@112000	A	prune interp d0.9-0.6	0.436	0.463	0.610	0.479	0.376	0.092	0.294	median-curve
6.9b@112000	A	prune extrap d0.55	2.121	1.939	2.224	2.061	0.191	1.272	2.198	strength-only
6.9b@112000	B	prune interp d0.9-0.6	0.131	0.106	0.235	0.140	0.111	0.095	0.294	median-curve
6.9b@112000	B	prune extrap d0.55	1.441	1.387	1.892	1.525	1.294	1.408	2.198	strength-only
Source	Prot.	Regime	full	no-D0	mean	median	zero	best		
1b@32000	A	quant ≥ 4 -bit	0.103	0.174	0.197	0.031	0.046	per-bit-median		
1b@32000	A	quant int3	1.050	2.648	2.903	1.438	3.108	full		
1b@32000	A	quant int5 (rule)	0.179	0.272	0.254	0.044	0.017	zero		
1b@32000	B	quant ≥ 4 -bit	0.028	0.032	0.027	0.026	0.046	per-bit-median		
1b@32000	B	quant int3	0.606	1.004	0.627	0.803	3.108	full		
1b@32000	B	quant int5 (rule)	0.025	0.013	0.035	0.034	0.017	noD0		
1b@112000	A	quant ≥ 4 -bit	0.071	0.111	0.183	0.036	0.063	per-bit-median		
1b@112000	A	quant int3	0.676	3.805	0.802	1.231	5.777	full		
1b@112000	A	quant int5 (rule)	0.151	0.138	0.251	0.035	0.025	zero		
1b@112000	B	quant ≥ 4 -bit	0.038	0.042	0.038	0.039	0.063	per-bit-mean		
1b@112000	B	quant int3	1.120	1.981	3.106	3.472	5.777	full		
1b@112000	B	quant int5 (rule)	0.046	0.019	0.027	0.025	0.025	noD0		
6.9b@32000	A	quant ≥ 4 -bit	0.253	0.317	0.190	0.034	0.033	zero		
6.9b@32000	A	quant int3	3.973	5.103	3.300	1.835	2.711	per-bit-median		
6.9b@32000	A	quant int5 (rule)	0.373	0.457	0.237	0.067	0.029	zero		
6.9b@32000	B	quant ≥ 4 -bit	0.033	0.050	0.029	0.025	0.033	per-bit-median		
6.9b@32000	B	quant int3	0.489	1.792	0.174	0.407	2.711	per-bit-mean		
6.9b@32000	B	quant int5 (rule)	0.046	0.052	0.058	0.057	0.029	zero		
6.9b@112000	A	quant ≥ 4 -bit	0.271	0.183	0.155	0.042	0.061	per-bit-median		
6.9b@112000	A	quant int3	3.873	3.550	0.430	1.896	6.442	per-bit-mean		
6.9b@112000	A	quant int5 (rule)	0.402	0.247	0.253	0.033	0.027	zero		
6.9b@112000	B	quant ≥ 4 -bit	0.048	0.060	0.042	0.045	0.061	per-bit-mean		
6.9b@112000	B	quant int3	1.294	4.033	3.770	4.137	6.442	full		
6.9b@112000	B	quant int5 (rule)	0.043	0.025	0.025	0.023	0.027	per-bit-median		

Table 10: P1-v2 new-source frozen prospectives (MAE over three capabilities, nats/token). Protocol A fits on the full nine-state development panel; protocol B on step $\leq 64k$ only, so its 112k rows are training-stage extrapolations. 1B sources are in-range sizes and 6.9B is a $\sim 5\times$ size extrapolation. Predictions were committed before measurement; per-capability and per-point values are in the results ledger.